Thinking in parallel

How to design algorithms for multi-processor systems

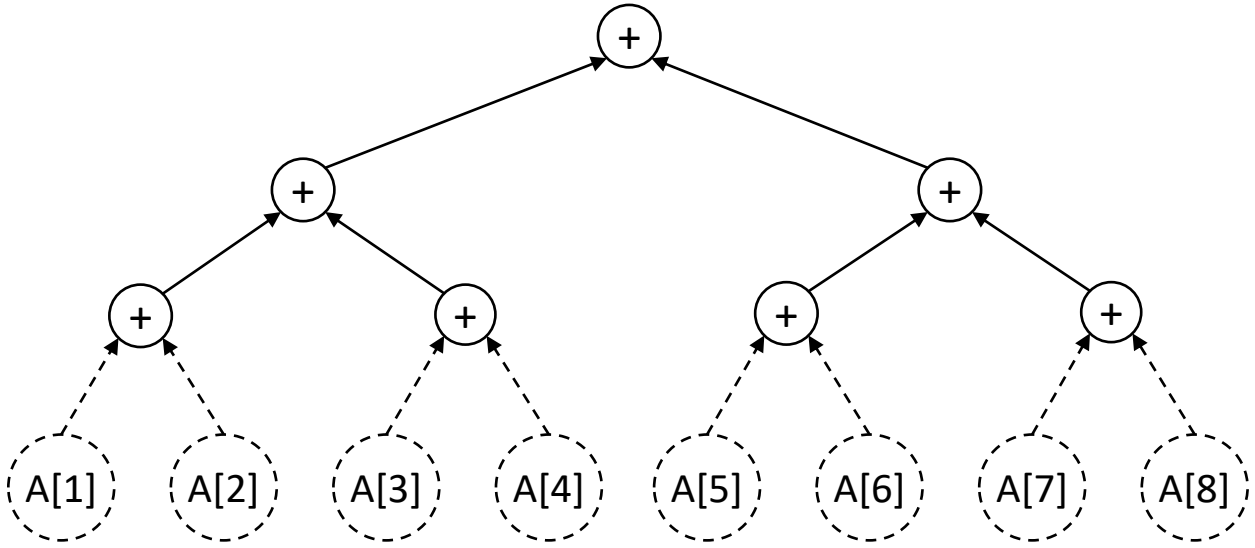
Basic techniques

Basic techniques: Balanced trees

- Uses the balanced binary tree scheme
- To solve the problem, the balanced binary tree is traversed forward and backward

EXAMPLE: Sum of all n elements of an array A
/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

```
1.  for 1 ≤ i ≤ n pardo B[i] ← A[i]
2.  for h = 1 to log2 n do
3.     for 1 ≤ i ≤ n/2^h pardo
4.        B[i] ← B[2i – 1] + B[2i]
5.  S ← B[1]
```



Balanced binary tree scheme

Balanced trees: Prefix Sum

PREFIX SUM

Input: An array A of n elements

Output: An array B of n elements, where $B[i] = \sum_{j=1}^{i} A[j]$

EXAMPLE:

A	3	4	-1	5	-3	2	3	-8
B	3	7	6	11	8	10	13	5

AN $O(n)$ TIME SEQUENTIAL ALGORITHM

```
B[1] ← A[1]
for i = 2 to n do
    B[i] ← B[i – 1] + A[i]
```

it is asymptotically optimal, …

… but it is not parallelizable

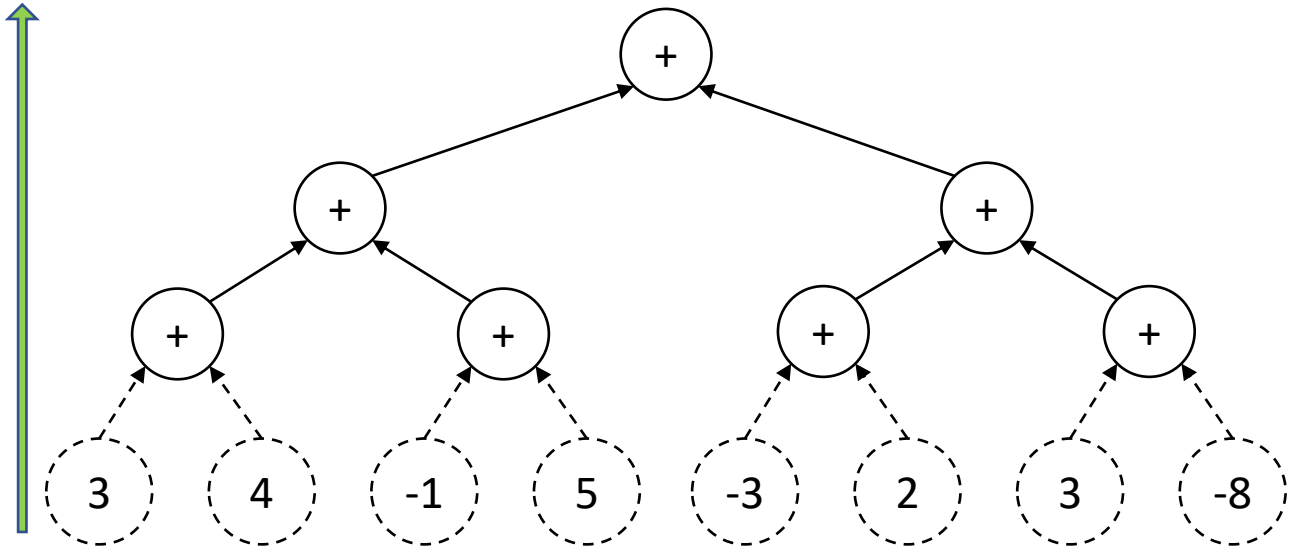
Applications of prefix sum

- perform lexical analysis
- evaluate polynomials
- solve tridiagonal linear systems
- delete marked elements of an array
- dynamically allocate processors
- label components in two-dimensional images
- implement some sorting algorithms like radix sort and quicksort
- implement some operations on trees (e.g., finding the depth of a node)

A solution

OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes



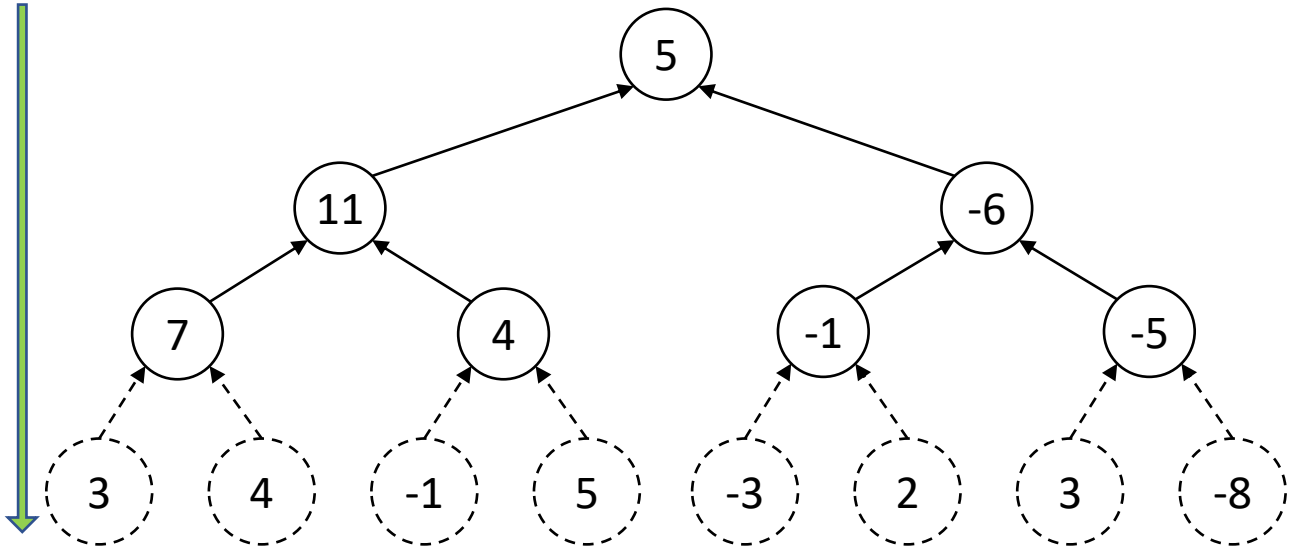
A	3	4	-1	5	-3	2	3	-8

A solution

OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes

$O(\log n)$ time and $O(n)$ work for forward traversal



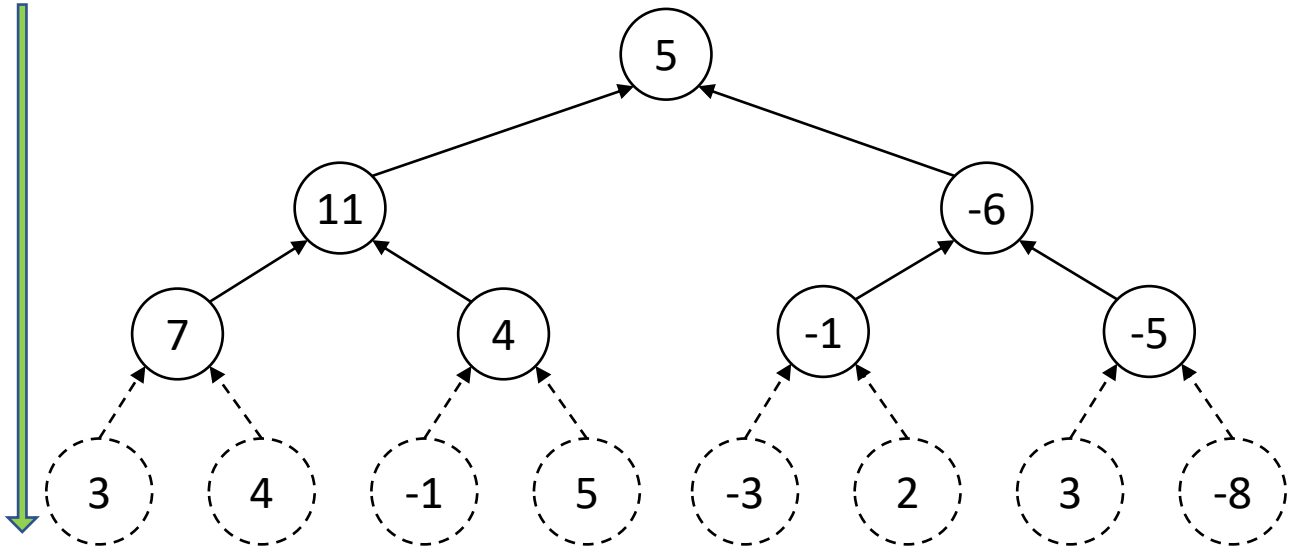
A								
	3	4	-1	5	-3	2	3	-8

A solution

OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes
- In the backward traversal of the tree, the prefix sum of the data stored at a specific level of the tree is computed

$O(\log n)$ time and $O(n)$ work for forward traversal



A							
3	4	-1	5	-3	2	3	-8

A solution

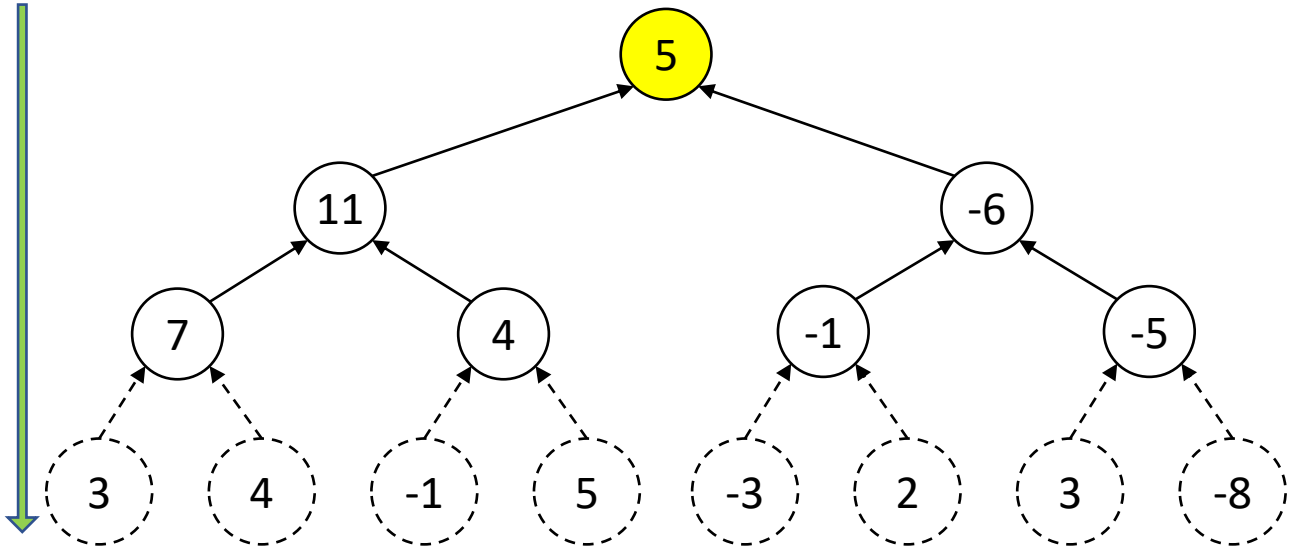
OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes
- In the backward traversal of the tree, the prefix sum of the data stored at a specific level of the tree is computed

$O(\log n)$ time and $O(n)$ work for forward traversal

RULES:

- First node does not change



A								
	3	4	-1	5	-3	2	3	-8

A solution

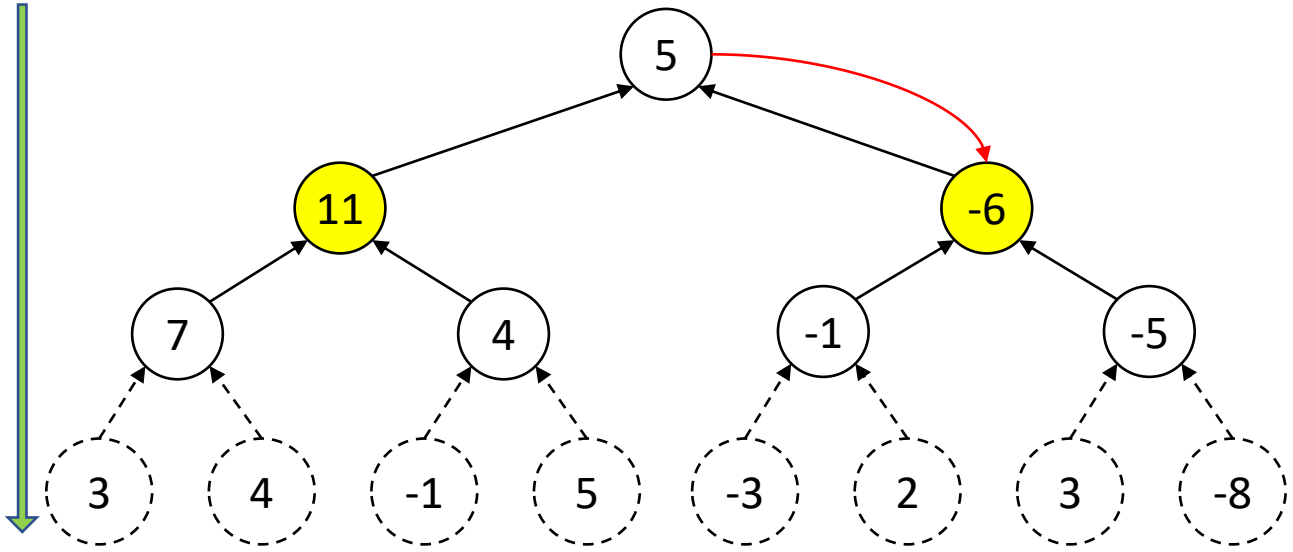
OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes
- In the backward traversal of the tree, the prefix sum of the data stored at a specific level of the tree is computed

$O(\log n)$ time and $O(n)$ work for forward traversal

RULES:

- First node does not change
- A right child inherits the value from its parent



A							
3	4	-1	5	-3	2	3	-8

A solution

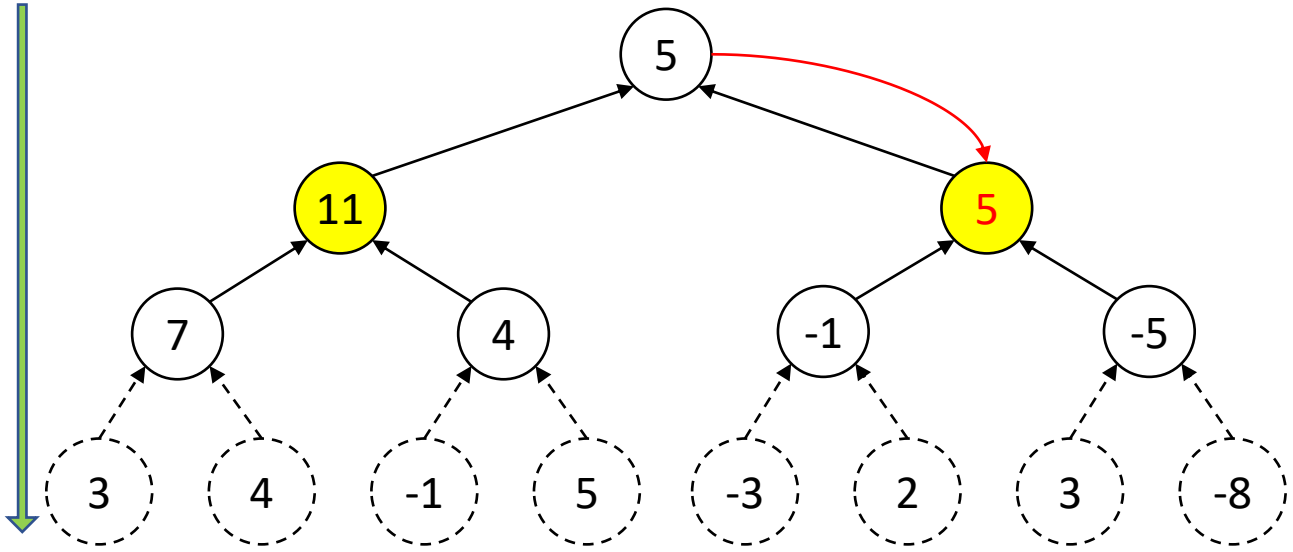
OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes
- In the backward traversal of the tree, the prefix sum of the data stored at a specific level of the tree is computed

$O(\log n)$ time and $O(n)$ work for forward traversal

RULES:

- First node does not change
- A right child inherits the value from its parent



A							
3	4	-1	5	-3	2	3	-8

A solution

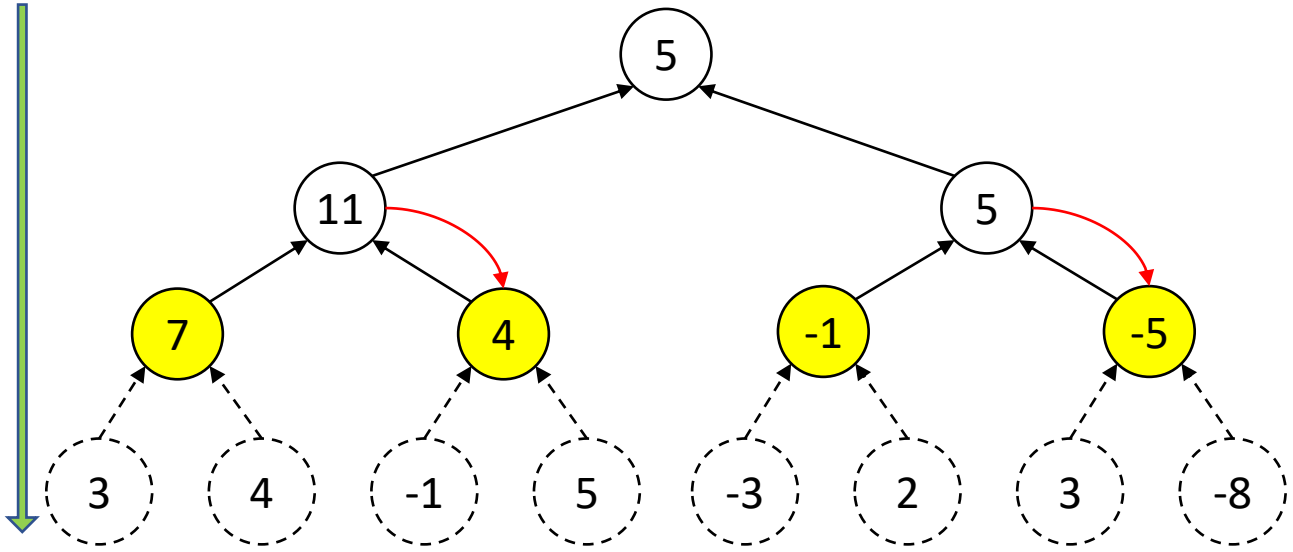
OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes
- In the backward traversal of the tree, the prefix sum of the data stored at a specific level of the tree is computed

$O(\log n)$ time and $O(n)$ work for forward traversal

RULES:

- First node does not change
- A right child inherits the value from its parent



A								
	3	4	-1	5	-3	2	3	-8

A solution

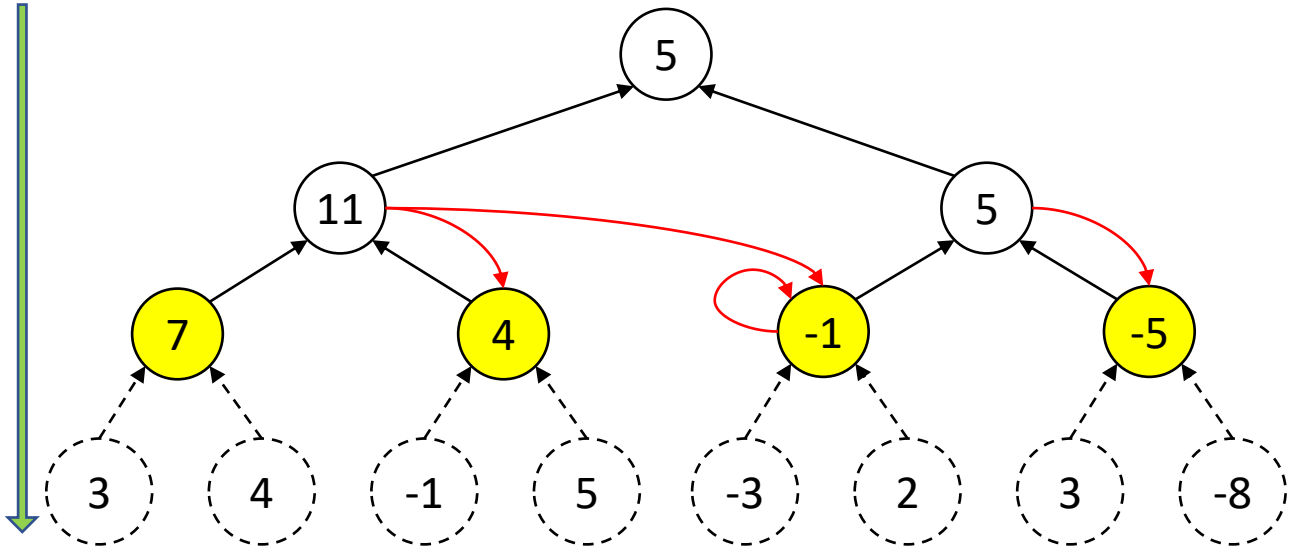
OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes
- In the backward traversal of the tree, the prefix sum of the data stored at a specific level of the tree is computed

$O(\log n)$ time and $O(n)$ work for forward traversal

RULES:

- First node does not change
- A right child inherits the value from its parent
- A left child (but the leftmost one) is modified by adding its parent predecessor to it



A							
3	4	-1	5	-3	2	3	-8

A solution

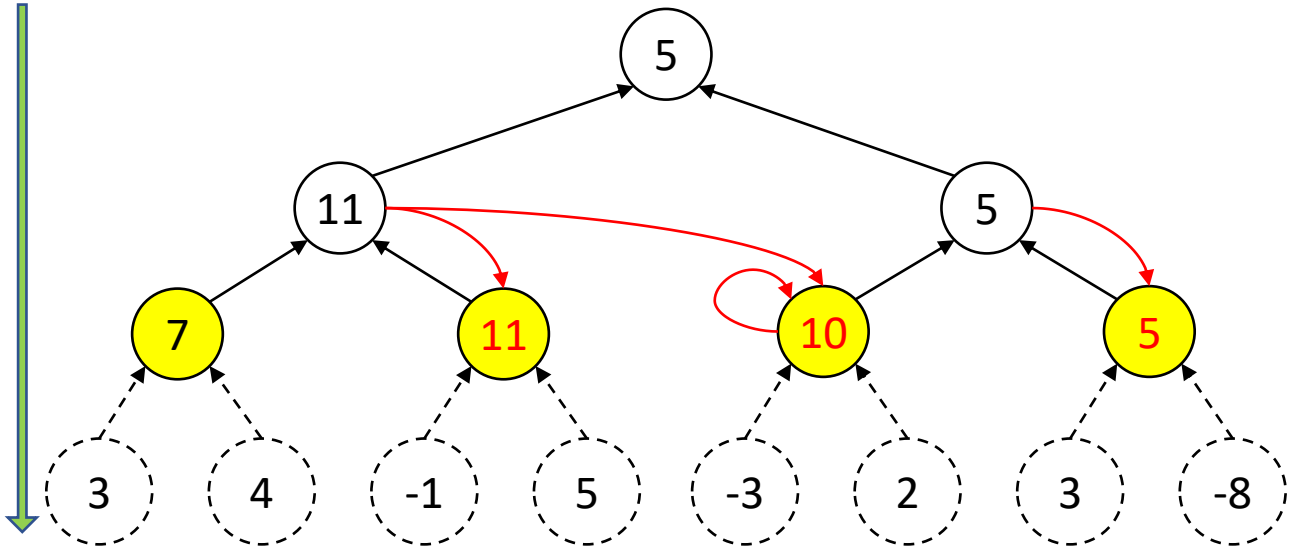
OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes
- In the backward traversal of the tree, the prefix sum of the data stored at a specific level of the tree is computed

$O(\log n)$ time and $O(n)$ work for forward traversal

RULES:

- First node does not change
- A right child inherits the value from its parent
- A left child (but the leftmost one) is modified by adding its parent predecessor to it



A	3	4	-1	5	-3	2	3	-8

A solution

OUR SOLUTION: We use the balanced binary tree scheme (i-th leaf of the tree represents $B[i]$)

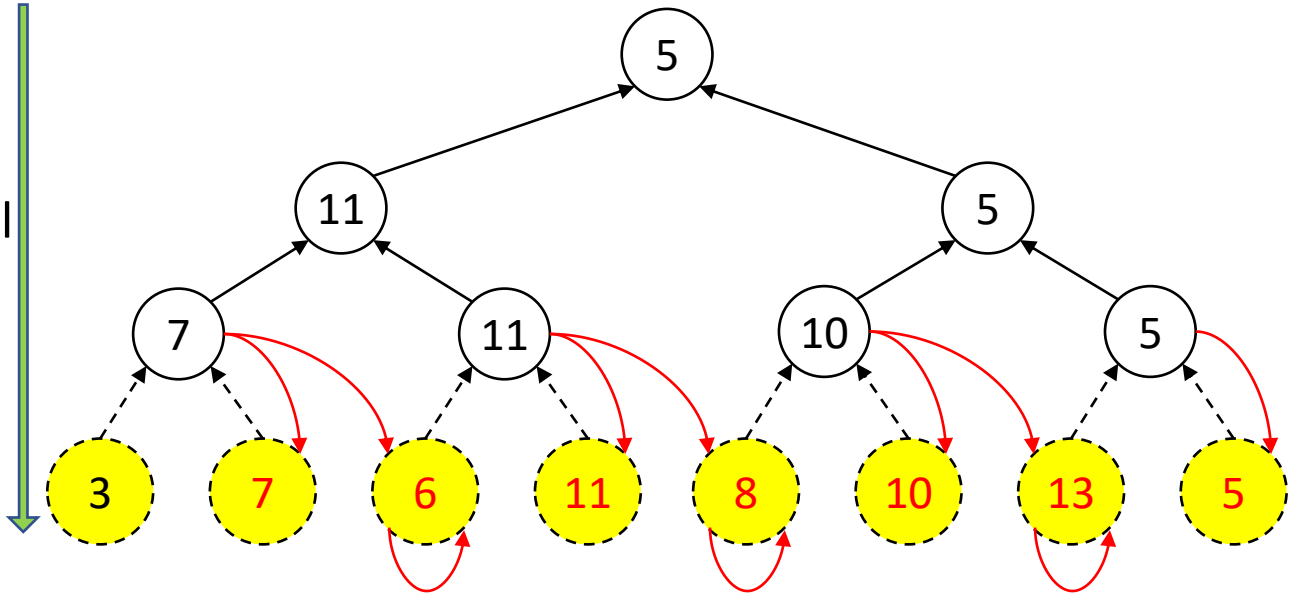
- In the forward traversal of the tree, we compute the overall sum of the elements stored at the leaves of the subtree rooted at the explored nodes
- In the backward traversal of the tree, the prefix sum of the data stored at a specific level of the tree is computed

$O(\log n)$ time and $O(n)$ work for forward traversal

$O(\log n)$ time and $O(n)$ work for backward traversal

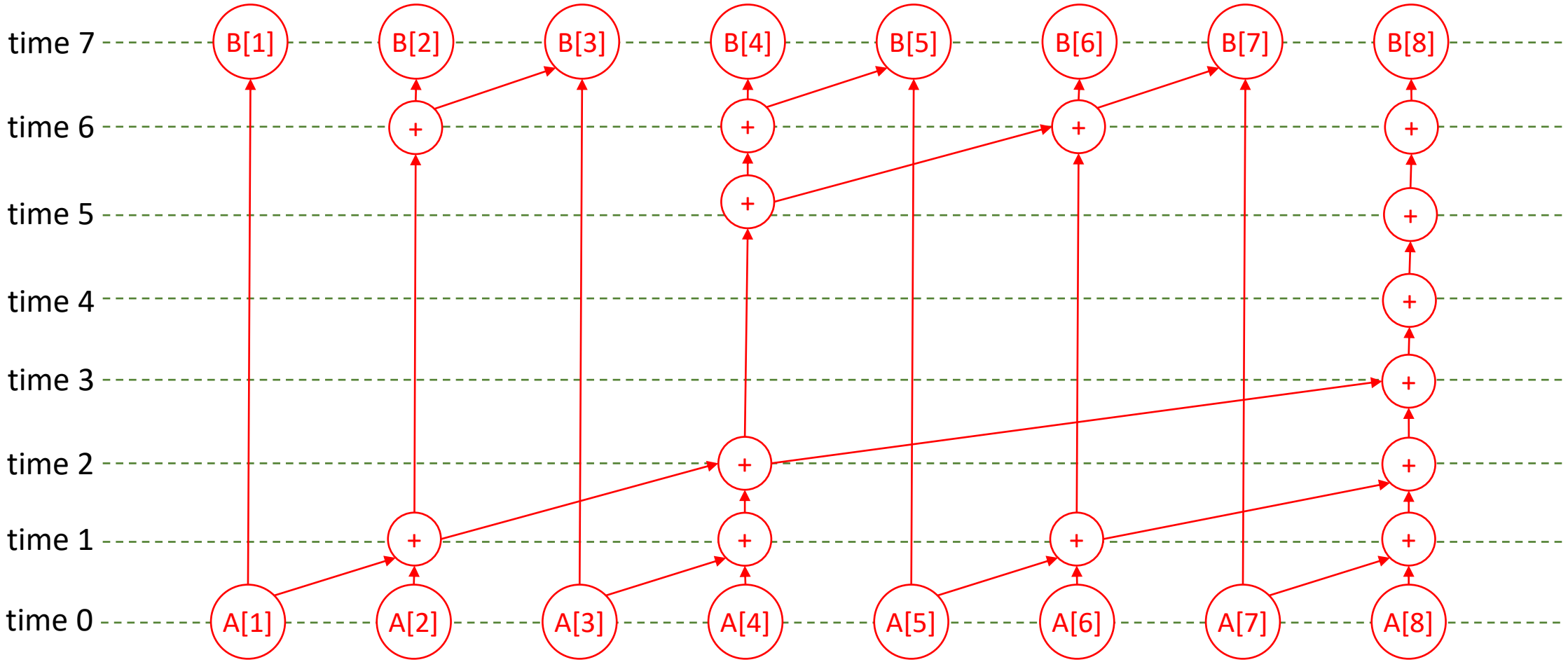
RULES:

- First node does not change
- A right child inherits the value from its parent
- A left child (but the leftmost one) is modified by adding its parent predecessor to it

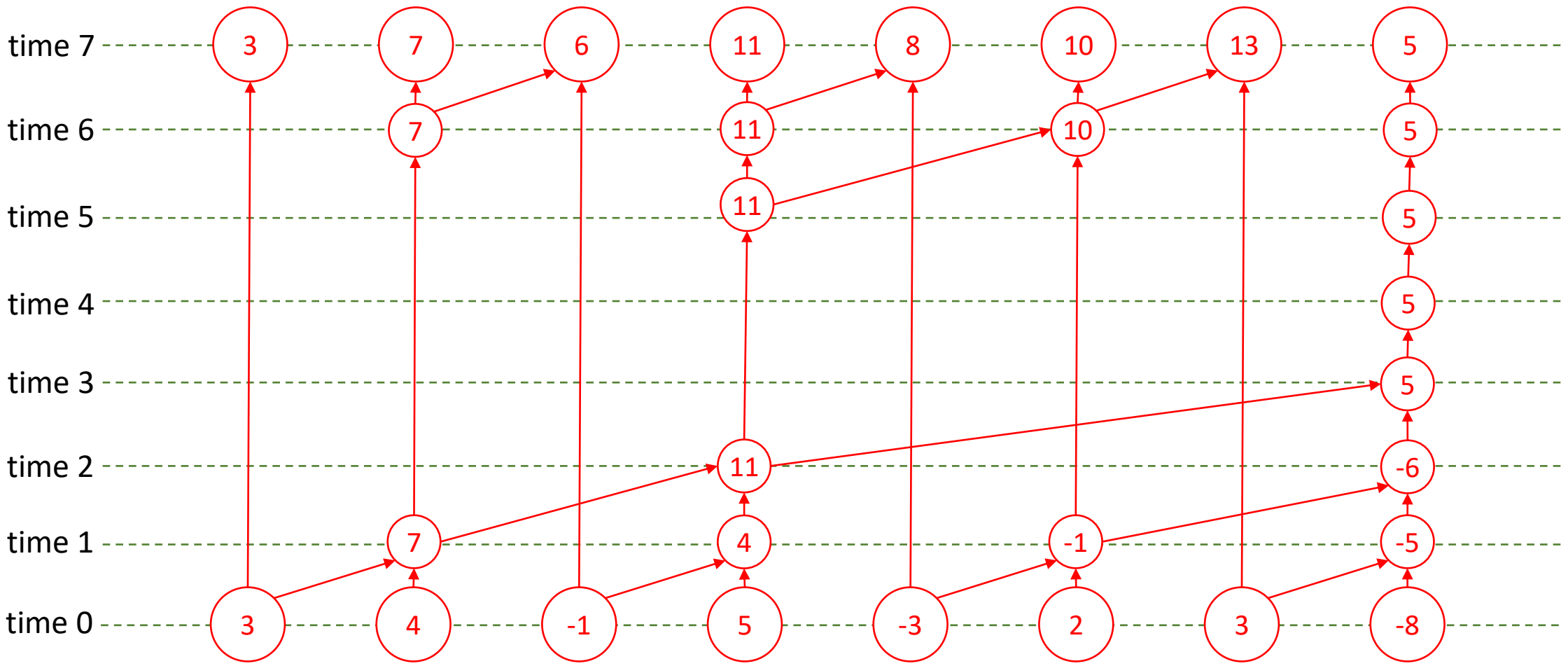


A	3	4	-1	5	-3	2	3	-8

The forward and backward traversals as a DAG



The forward and backward traversals as a DAG



Implementation with a recursive algorithm

ALGORITHM Prefix_Sum(n, A) /* it computes the prefix sum; we assume that $n=2^k$ */

1. **if** n = 1 **then** B[1] ← A[1]; **exit**
2. **for** $1 \leq i \leq n/2$ **pardo** X[i] ← A[2i – 1] + A[2i]
3. Y ← Prefix_Sum(n/2, X)
4. **for** $1 \leq i \leq n$ **pardo**
5. i = 1 : B[1] ← A[1]
6. i even : B[i] ← Y[i/2]
7. i>1 odd : B[i] ← Y[(i – 1)/2] + A[i]

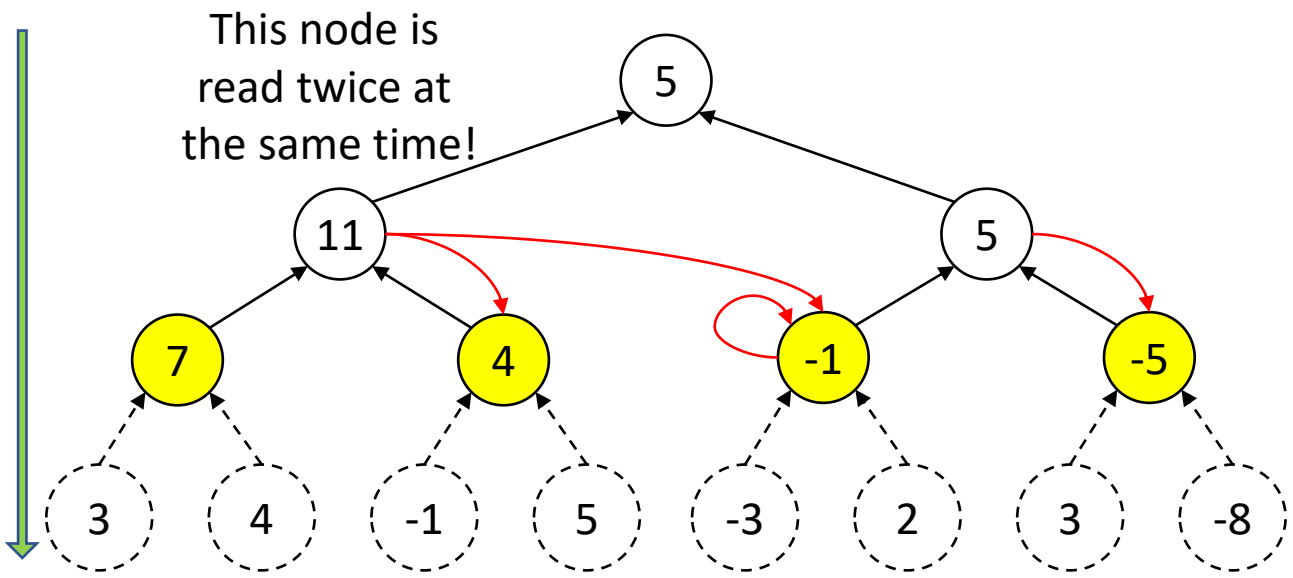
- Time $T(n) = T\left(\frac{n}{2}\right) + O(1) = O(\log n)$
- Work $W(n) = W\left(\frac{n}{2}\right) + O(n) = O(n)$ (work optimal)
- Concurrent writes on same variable?
  - NO
- Concurrent reads on same variable?
  - YES

This is a CREW PRAM algorithm

An example of a concurrent read

RULES:

- First node does not change
- A right child inherits the value from its parent
- A left child (but the leftmost one) is modified by adding its parent predecessor to it



A	3	4	-1	5	-3	2	3	-8

Can we avoid concurrent reads?

PREFIX SUM
Input: An array A of n elements
Output: An array B of n elements, where $B[i] = \sum_{j=1}^{i} A[j]$

ALGORITHM Prefix_Sum(n, A) /* it computes the prefix sum; we assume that $n=2^k$ */

1. **if** n = 1 **then** B[1] ← A[1]; **exit**
2. **for** 1 ≤ i ≤ n/2 **pardo** X[i] ← A[2i – 1] + A[2i]
3. Y ← Prefix_Sum(n/2, X)
4. **for** 1 ≤ i ≤ n **pardo**
5. i = 1 : B[1] ← A[1]
6. i even : B[i] ← Y[i/2]
7. i>1 odd : B[i] ← Y[(i – 1)/2] + A[i]

- Time $T(n) = T\left(\frac{n}{2}\right) + O(1) = O(\log n)$
- Work $W(n) = W\left(\frac{n}{2}\right) + O(n) = O(n)$ (work optimal)
- Concurrent writes on same variable?
  - NO
- Concurrent reads on same variable?
  - YES

This is a CREW PRAM algorithm

Can we avoid concurrent reads on same variable?

Avoiding concurrent reads with the COPY TRICK

PREFIX SUM
Input: An array A of n elements
Output: An array B of n elements, where $B[i] = \sum_{j=1}^{i} A[j]$

ALGORITHM Prefix_Sum(n, A) /* it computes the prefix sum; we assume that $n=2^k$ */

1. **if** n = 1 **then** B[1] ← A[1]; **exit**
2. **for** 1 ≤ i ≤ n/2 **pardo** X[i] ← A[2i – 1] + A[2i]
3. Y ← Prefix_Sum(n/2, X)
4. **for** 1 ≤ i ≤ n/2 **pardo** Z[i] ← Y[i]
5. **for** 1 ≤ i ≤ n **pardo**
6. i = 1 : B[1] ← A[1]
7. i even : B[i] ← Y[i/2]
8. i >1 odd : B[i] ← Z[(i – 1)/2] + A[i]

- Time $O(\log n)$
- Work $O(n)$ (work optimal)
- Concurrent reads on same variable?
  - NO
- Concurrent writes on same variable?
  - NO

This is an EREW PRAM algorithm

Formal proof of the main theorem

THEOREM: The Prefix_Sum algorithm runs on a EREW PRAM and computes the prefix sums of n elements in time $T(n) = O(\log n)$ and work $W(n) = O(n)$.

PROOF (only correctness): By induction on k, where $n = 2^k$.
BASE CASE $k = 0$: correctness follows by Step 1.
INDUCTIVE CASE $k + 1$: Step 5 guarantees the correctness for the prefix sum with index $i = 1$. By the induction hypothesis, the array Y of size 2^{k-1} computed during Step 3 correctly contains the prefix sum of array X. In particular, as $X[j] = A[2j-1] + A[2j]$ (Step 2), we have

$$Y[i] = \sum_{j=1}^{i} X[j] = \sum_{j=1}^{i} (A[2j-1] + A[2j]) = \sum_{j=1}^{2i} A[j]$$

Therefore, $Y[i]$ contains the prefix sum of A corresponding to the even index $2i$.
So, Step 6 is correct. Moreover, Step 7 is also correct as the prefix sum of the first i elements of A, for all odd values of $i > 1$, is equal to $Y[(i-1)/2] + A[i]$.

Can we do better?

THEOREM: The Prefix_Sum algorithm runs on a EREW PRAM and computes the prefix sums of n elements in time $T(n) = O(\log n)$ and work $W(n) = O(n)$.

THEOREM (without proof): Any CREW PRAM algorithm that solves Prefix Sum problem requires time $T(n) = \Omega(\log n)$.

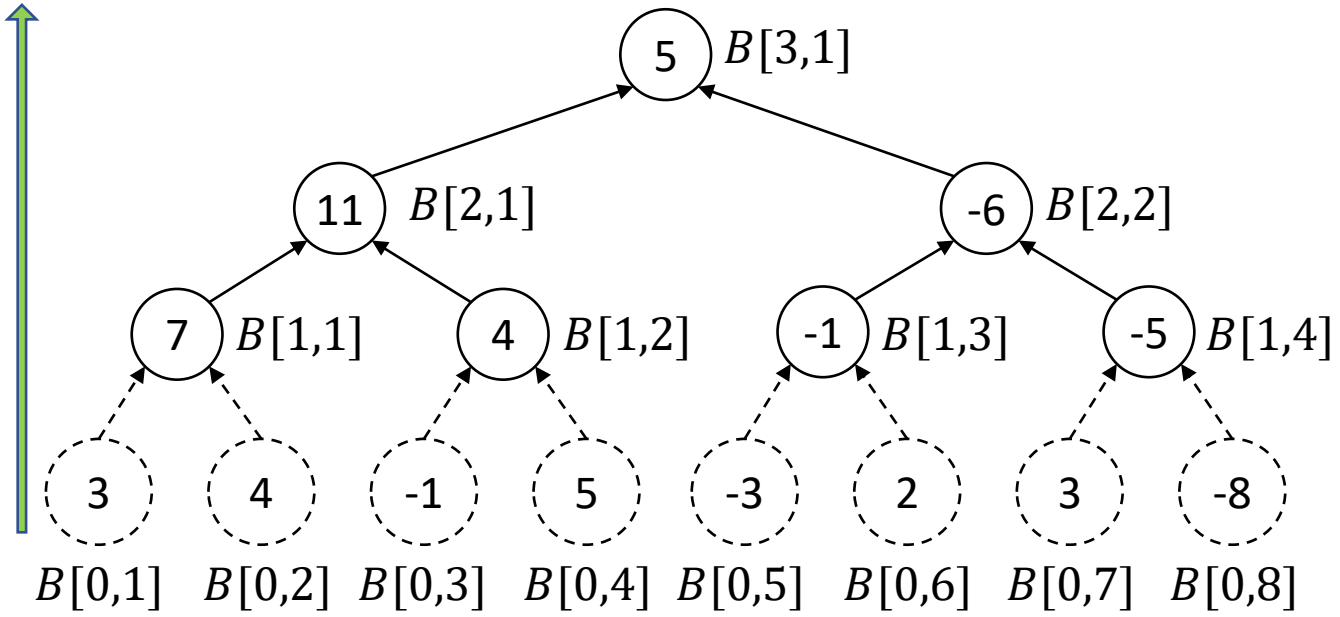
OBSERVATION: The Prefix_Sum algorithm is work-time optimal.

A non-recursive solution

NON-RECURSIVE SOLUTION:

- Double the length of the array B (instead of n it is $2n$)
- For the sake of simplicity, think of B as a matrix (e.g., $B[j,i]$ is the entry $i + \sum_{h=1}^{j} 2^{k+1-h}$)

RULE FOR THE FORWARD TRAVERSAL:
$B[j,i] \leftarrow B[j-1,2i-1] + B[j-1,2i]$



A	3	4	-1	5	-3	2	3	-8

A non-recursive solution

NON-RECURSIVE SOLUTION:

- Double the length of the array B (instead of n it is $2n$)
- For the sake of simplicity, think of B as a matrix (e.g., $B[j,i]$ is the entry $i + \sum_{h=1}^{j} 2^{k+1-h}$)

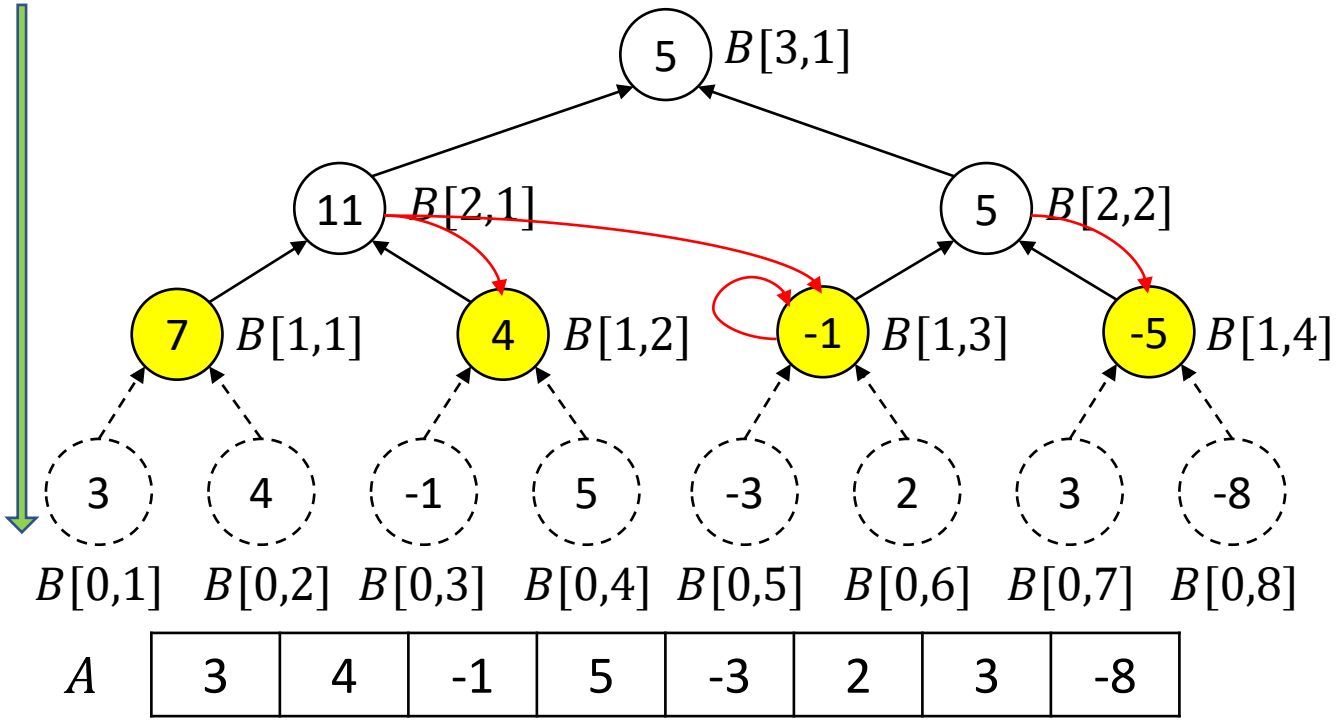
RULES FOR THE BACKWARD TRAVERSAL:

- First node does not change: do nothing
- A right child inherits the value from its parent

$$B[j,i] \leftarrow B[j+1, i/2]$$

- A left child (but the leftmost one) is modified by adding its parent predecessor to it

$$B[j,i] \leftarrow B[j+1, (i-1)/2] + B[j,i]$$



Implementation of the non-recursive solution

ALGORITHM Prefix_Sum mancano gli argomenti n e A

/* it computes the prefix sum in B[0,i], $1 \leq i \leq n$; we assume that $n=2^k$ */

1. **for** $1 \leq i \leq n$ **pardo** B[0,i] ← A[i]

2. **for** j=1 **to** $\log_2$ n **do** // FORWARD TRAVERSAL

3. **for** $1 \leq i \leq n/2^j$ **pardo**

4. B[j, i] ← B[j – 1, 2i – 1] + B[j – 1, 2i] errore

5. **for** j=1 **to** $\log_2$ n **do** // BACKWARD TRAVERSAL

6. **for** $1 \leq i \leq n/2^j$ **pardo**

7. i even : B[j, i] ← B[j + 1, i/2]

8. i > 1 odd : B[j, i] ← B[j + 1, (i – 1)/2] + B[j, i]

può essere evitato

- Time $O(\log n)$
- Work $O(n)$ (work optimal)
- Concurrent reads/writes on same variable?
- YES, there are concurrent reads on same variable in the current pseudocode, but we can use the COPY TRICK as we did in the recursive solution

This is an EREW PRAM algorithm

1. for $1 \le i \le n$ pardo $B[0,1] \leftarrow A[i]$
2. for $j=1$ to $\log_2 n$ do
3. for $1 \le i \le \frac{n}{2^j}$ pardo
4. $B[j,i] \leftarrow B[j-1, 2i-1] + B[j-1, 2i]$
5. for $j = \log_2 n - 1$ to 0
6. for $1 \le i \le \frac{n}{2^j}$ pardo
7. if $(i \% 2 = 0)$ then
8. $B[j,i] \leftarrow B[j+1, \frac{i}{2}]$
9. else if $(i > 1 \&\& i \% 2 != 0)$ then
10. $B[j,i] \leftarrow B[j+1, \frac{i-1}{2}] + B[j,i]$

$\log_2 n$ included $(1 \le j \le \log_2 n)$

per consistenza

$(\log_2 n - 1 \le j \le 0)$

compreso → deve scendere al livello 0 perché le foglie rappresentano l'array delle prefix sum

Considerazioni:

- Il ciclo a 5. deve partire dal livello $\log_2 n - 1$ (livello sotto la radice) e scendere a 0 compreso (foglie), altrimenti che backward traversal stiamo facendo?
- Il ciclo a 6. può partire da 2 e ci evitiamo il problema di escludere il nodo più a sinistra del livello j, quindi l'else if può diventare un else
- Inconsistenze, gli array sono 1-based, il livello dell'albero 0 based, consiglio di riscrivere tutti gli algoritmi in 0-based

ATTENZIONE:

4. $B[j,i] \leftarrow B[j-1, 2i-1] + B[j-1, 2i]$

Se $i=0$ in una versione 0-based, allora $2i-1 = 2 \cdot 0 - 1 = -1$

=> Segmentation fault

Allora o considererei il livello delle foglie il livello 1 (per consistenza) o lascerei così in modo inconsistente

ORE PASSATE A DEBUGGARE: 3

Check you understanding

Let A be an array of $n = 2^k$ elements, where k is an integer. Develop an EREW PRAM work-time optimal algorithm that computes the prefix sum of A *in place*, i.e., without using additional shared variables.

Check you understanding

Let A be an array of n elements and let J be an array of s integers such that $0 = J[1] < J[2] < \cdots < J[s] = n$. Consider the problem of computing an array B of n elements such that, for every $i \in \{1, \ldots, s-1\}$ and every $J[i] < j \leq J[i+1]$, $B[j] = A[J[i+1]]$. Develop a work-optimal algorithm whose running time is $O(\log n)$.

EXAMPLE:

A	3	7	5	4	2	6	1	8

J	0	2	5	8

B	7	7	2	2	2	8	8	8

Check you understanding

Suppose we are given a set of n elements stored in an array A together with an array L such that $L[i] \in \{1,2,\dots,k\}$ represents the label of element $A[i]$, where k is a constant. Develop a work-optimal EREW PRAM algorithm with time $O(\log n)$ that stores all the elements of A with label 1 into the upper part of A while preserving their initial ordering, followed by the elements labeled 2 with the same initial ordering, and so on.

EXAMPLE: $k = 3$

A	3	7	5	4	2	6	1	8
L	1	3	2	3	1	3	2	1
new A	3	2	8	5	1	7	2	6